



2018

A short introduction to Python

Last update: July 2019



Aniel Bhulai

Vrije Universiteit Amsterdam

Contents

1. Introduction.....	7
2. Overview.....	8
3. Reserved keywords.....	9
4. Operators.....	10
4.1. Arithmetic operators	10
4.2. Assignment operators.....	11
4.3. Comparison (relational) operators	12
4.4. Logical operators	12
4.5. Identity operators.....	13
4.6. Bitwise operators.....	13
4.7. Membership operators.....	14
5. Strings	15
6. Escape characters	16
7. Numbers	17
7.1. Numerical data types.....	17
7.2. Number type conversion	18
7.3. Mathematical functions.....	19
7.4. Trigonometric functions	20
7.5. Random number functions	21
8. Data types.....	22
9. Variables	23
10. Lists.....	24
10.1. Creating lists.....	24
10.2. Accessing values in lists.....	24
10.3. Updating lists	25
10.4. Deleting values from lists	25
10.5. Basic list operations	26
10.6. Built-in list functions	26
10.7. Built-in list methods.....	27

11. Tuples	28
11.1. Creating tuples.....	28
11.2. Accessing values in tuples	28
11.3. Updating tuples.....	29
11.4. Deleting values from tuples	29
11.5. Basic tuple operations.....	30
11.6. Built-in tuple functions.....	30
12. Dictionary	31
12.1. Creating dictionaries	31
12.2. Accessing values in dictionaries	31
12.3. Updating dictionaries.....	32
12.4. Deleting dictionaries	32
12.5. Built-in dictionary functions	33
12.6. Built-in dictionary methods.....	34
13. Decision-making.....	35
13.1. if-statement	35
13.2. if...else-statement	36
13.3. if...elif...else-statement.....	37
13.4. Nested if-statement	38
14. Loops	39
14.1. While-loop	39
14.2. For-loop	40
14.3. Nested loop.....	42
14.4. Loop control statements	42
14.4.1. Break statement	42
14.4.2. Continue statement	43
14.4.3. Pass statement.....	43
15. Functions	45
15.1. Creating a function.....	45

15.2.	Calling a function	45
15.3.	Parameters and arguments.....	45
15.4.	Return values	46
15.5.	Parameters vs arguments	47
16.	Modules	48
16.1.	Creating modules	48
16.2.	Using modules.....	48
16.2.1.	Import statement.....	48
16.2.2.	Import...from statement	48
16.3.	Variables in modules	49
16.4.	Renaming modules	50
16.5.	Using the dir() function	50
17.	Date & Time	52
18.	Files I/O.....	55
18.1.	Writing to standard output	55
18.2.	Reading from standard input	56
18.3.	Opening a file	56
18.4.	Closing a file	58
18.5.	Reading a file.....	58
18.5.1.	Read the whole file	58
18.5.2.	Read a part of a file.....	59
18.5.3.	Read a few lines of a file	59
18.5.4.	Looping through a file	59
18.6.	Writing a file.....	59
18.7.	Creating a file	60
18.8.	Deleting a file or a directory.....	60
18.9.	Directory methods	61
Index		62
Acknowledgements.....		63

1. Introduction

Python is a general-purpose object-oriented, interactive and interpreted programming language with high-level programming capabilities. To learn the Python programming language you do not need any previous programming knowledge or skills. However, the basic understanding of any other programming language will help you to understand the Python programming concepts quickly. In this tutorial we will give you a short introduction to Python (version 3) in order to give you a tool to analyze and to design algorithms. So, this tutorial is in no way complete. Please, refer to the official Python documentation at <http://docs.python.org> if you need more information.

2. Overview

Python is designed to be highly readable and is a great language for beginners. It uses English keywords frequently. As mentioned in the introduction, Python is an interpreted programming language. The Python source code is compiled to bytecode as a **.pyc** file, and this bytecode can be interpreted. The Python interpreter can be used in two modes: interactive and script mode.

In the interactive mode, the Python script can be executed directly to the Python prompt without passing the script file to the interpreter. The interactive mode is useful when dealing with small pieces of code as you can type and execute them directly at the Python prompt.

In the script mode, the Python script file is stored with the **.py** extension. The interpreter is then used to execute the contents of the script file. The name of the script file is passed to the interpreter to be executed. The script mode is useful when the code is more than 4 lines and when you want to use the code in future. Example 1 shows how to run the script file **HelloWorld.py** on a UNIX/LINUX system.

Example

```
python HelloWorld.py
```

Example 1: Passing a Python script file to the interpreter

3. Reserved keywords

Like in most programming language, the Python programming language has some reserved keywords which you cannot use as constant, variable or any other identifier names. All the Python reserved keywords contain lowercase letters only. Table 1 shows the Python reserved keywords.

as	continue	except	global	lambda	raise	yield
and	def	exec	if	not	return	
assert	del	finally	import	or	try	
break	elif	for	in	pass	while	
class	else	from	is	print	with	

Table 1: Reserved keywords in Python

4. Operators

Operators are special symbols that are used to perform mathematical or logical manipulations. *Operands* are the values or variables with which the operator is applied to. The values of operands can be manipulated by using the operators.

Example

$10 + 2 = 12$

Example 2: Operator and operands

In Example 2 we see two operands (i.e., 10 and 2) and a plus symbol (+) which is the operator. The plus symbol performs the addition. The output of the operation is 12.

The Python programming language supports the following types of operators:

- Arithmetic operators
- Assignment operators
- Comparison (relational) operators
- Logical operators
- Identity operators
- Bitwise operators
- Membership operators

4.1. Arithmetic operators

Arithmetic operators perform various arithmetic calculations like addition, subtraction, multiplication, division, exponent, etc. There are various methods for arithmetic calculation in Python. You can for example use the `eval()` function, declare variable & calculate, or call functions. Table 2 shows the arithmetic operators that are supported by Python.

Symbol	Operator name	Example	Description
+	Addition	$10 + 2 = 12$	This operator adds the values on either side of the operator.
-	Subtraction	$6 - 3 = 3$	This operator subtracts the right side value from the left side value.
*	Multiplication	$8 * 2 = 16$	This operator multiplies the values on both sides of the operator.
/	Division	$4 / 2 = 2$	This operator divides the left side value with the right side value.
%	Modulus	$7 \% 2 = 1$	This operator returns the remainder by dividing the left side value with right side value.

**	Exponent	$2^{**}3 = 8$	This operator calculates the exponential power.
//	Floor division	$9 // 4 = 2$	This operator calculates the result of $9/4$ and leaves the digits after the decimal point away.

Table 2: Arithmetic operators

4.2. Assignment operators

Python assignment operators are used for assigning the value of the right operand to the left operand. Table 3 shows the assignment operators that are supported by Python.

Symbol	Operator name	Example	Description
=	Equal	$11 + 2 = 13$	This operator calculates the sum of the value of the right side operand and the left side operand.
+=	Add AND	$p += 2$	Same as $p = p + 2$
-=	Subtract AND	$p -= 3$	Same as $p = p - 3$
*=	Multiply AND	$p *= 4$	Same as $p = p * 4$
/=	Division AND	$p /= 5$	Same as $p = p / 5$
%=	Modulus AND	$p \% = 4$	Same as $p = p \% 4$
**=	Exponent AND	$p ** = 3$	Same as $p = p ** 3$
//=	Floor division AND	$p //= 6$	Same as $p = p // 6$

Table 3: Assignment operators

4.3. Comparison (relational) operators

Comparison operators, also referred as relational operators, compare the values on either side of the operand and determine the relation between them. Table 4 shows the comparison operators that are supported by Python.

Symbol	Operator name	Example	Description
<code>==</code>	Double equal	<code>a == b</code>	TRUE if <i>a</i> is equal to <i>b</i> .
<code>!=</code>	Not equal to	<code>a != b</code>	TRUE if <i>a</i> is not equal to <i>b</i> .
<code><></code>	Not equal to	<code>a <> b</code>	TRUE if <i>a</i> is not equal to <i>b</i> .
<code>></code>	Greater than	<code>a > b</code>	TRUE if <i>a</i> is greater than <i>b</i> .
<code><</code>	Less than	<code>a < b</code>	TRUE if <i>a</i> is less than <i>b</i> .
<code><=</code>	Less than equal to	<code>a <= b</code>	TRUE if <i>a</i> is less than or equal to <i>b</i> .
<code>>=</code>	Greater than equal to	<code>a >= b</code>	TRUE if <i>a</i> is greater than or equal to <i>b</i> .

Table 4: Comparison (relational) operators

4.4. Logical operators

Table 5 shows the logical operators that are supported by Python.

Symbol	Operator name	Example	Description
<code>or</code>	Logical OR	<code>a or b</code>	TRUE if either <i>a</i> or <i>b</i> is TRUE .
<code>and</code>	Logical AND	<code>a and b</code>	TRUE if both <i>a</i> and <i>b</i> are TRUE .
<code>not</code>	Logical NOT	<code>not a</code>	TRUE if <i>a</i> is FALSE .

Table 5: Logical operators

4.5. Identity operators

Identity Operators are used to compare the memory location of two objects. Table 6 shows the identity operators that are supported by Python.

Symbol	Operator name	Example	Description
is	is	$a \text{ is } b$	TRUE if values on either side of the operator point to the same object and FALSE otherwise.
is not	is not	$a \text{ is not } b$	FALSE if the variables on either side of the operator point to the same object and TRUE otherwise.

Table 6: Identity operators

4.6. Bitwise operators

Table 7 shows the bitwise operators that are supported by Python.

Symbol	Operator name	Example	Description
&	Binary AND	$a \& b$	Operator copies a bit to the result if it exists in both operands.
	Binary OR	$a b$	Operator copies a bit if it exists in either operand.
^	Binary XOR	$a \wedge b$	Operator copies the bit if it is set in one operand but not both.
~	Binary 1s Complement	$\sim a$	Operator is unary and has the effect of ‘flipping’ bits.
<<	Binary Left Shift	$a \ll b$	The left operands value is moved left by the number of bits specified by the right operand.
>>	Binary Right Shift	$a \gg b$	The left operands value is moved right by the number of bits specified by the right operand.

Table 7: Bitwise operators

4.7. Membership operators

Membership operators test for membership in a sequence such as lists, strings or tuples. Table 8 shows the membership operators that are supported by Python.

Symbol	Operator name	Example	Description
in	in	a in b	TRUE if it finds a variable in the specified sequence and FALSE otherwise.
not in	not in	a not in b	TRUE if it does not finds a variable in the specified sequence and FALSE otherwise.

Table 8: Membership operators

5. Strings

A string is a list of characters represented in either double quote (") or single quote (').

Example

```
str = "Study algorithms."

print (str)           # Prints complete string
print (str[0])        # Prints 1st character of the string
print (str[4:10])     # Prints characters starting from the 5th to 10th
print (str[4:])       # Prints characters starting from the 5th
print (str * 2)       # Prints string two times
print (str + "Hello") # Prints concatenated string
```

Example 3: Strings

Example 3 shows a few examples of strings. We can take a subset of strings using the slice operator ([] and [:]) with index numbers. The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator. Table 9 shows the output of Example 3.

Output example

```
Study algorithms.
S
y algo
y algorithms.
Study algorithms.Study algorithms.
Study algorithms.Hello
```

Table 9: Output of Example 3

6. Escape characters

The Python programming language has some special characters which are used for special purposes. These characters are represented by a backslash followed by character(s). They can be interpreted using both single and double quote. Table 10 shows the escape characters in Python.

Backslash notation	Hexadecimal character	Description
\a	0x07	Bell or alert
\b	0x08	Backspace
\cx		Control-x
\C-x		Control-x
\e	0x1b	Escape
\f	0x0c	Form feed
\M-\C-x		Meta-Control-x
\n	0x0a	Newline
\nnn		Octal notation, n is in the range 0-7
\r	0x0d	Carriage return
\s	0x20	Space
\t	0x09	Tab
\v	0x0b	Vertical tab
\x		Character x
\xnn		Hexadecimal notation, n is in the range 0-9, a-f, or A-F

Table 10: Escape characters in Python

7. Numbers

Python supports integers, floating point numbers and complex numbers. These are number-based data types that store various types of numeric values. Number objects are created when we assign a value to them, see Example 4.

Example

```
var1 = 20
var2 = 7.3
```

Example 4: Creating number objects

The reference to a number object can be deleted by using the **del** statement, see Example 5.

Example

```
del var1
del var1, var2
```

Example 5: Syntax of the del statement

7.1. Numerical data types

Table 11 shows the four numerical data types that are supported by Python.

Numerical data types	Examples	Description
complex (complex numbers)	7.15j 97.j 4.78e-8j	Complex numbers are of the form a + bJ , where a and b are floating point numbers and J (or j) represents the square root of -1 (imaginary number).
float (floating point real values)	16.13 -34.78 2.5e100	Floating points represent real numbers. They are written with a decimal point dividing the integer and fractional parts. Floats may also be written in scientific notation. The E or e indicates the power of 10 (e.g., $3.4e2 = 3.4 \times 10^2 = 340$).
int (signed integers)	10 -89 -0x260	Signed integers are positive or negative whole numbers with no decimal point.
long (long integers)	62835472L -0x19453L -067674545L	Long integers are integers of unlimited size. They are written like integers and followed by an uppercase or lowercase L.

Table 11: Numerical data types in Python

7.2. Number type conversion

Python converts numbers internally in an expression that contains mixed numerical data types to a common type for evaluation. We can force a number from one type to another type in Python. This is sometimes necessary to satisfy the requirements of an operator or function parameter.

Number type conversion	Description
<code>int(x)</code>	Is used to convert value <code>x</code> to an integer.
<code>long(x)</code>	Is used to convert value <code>x</code> to a long integer.
<code>float(x)</code>	Is used to convert value <code>x</code> to a floating point value.
<code>complex(x)</code>	Is used to convert value <code>x</code> to a complex number with real part <code>x</code> and imaginary part as 0.
<code>complex(x, y)</code>	Is used to convert values <code>x</code> and <code>y</code> to a complex number with real part <code>x</code> and imaginary part <code>y</code> .

Table 12: Number type conversion

7.3. Mathematical functions

Table 13 shows the mathematical functions in Python. Note that some functions are not accessible directly. We need to import the math module first in order to call the function using the math static object, e.g., `math.log(10)`.

Mathematical function	Description	Example	Output example
<code>abs(<i>x</i>)</code>	The absolute value of <i>x</i> : the positive distance between <i>x</i> and zero.	<code>abs(-30)</code> <code>abs(202.19)</code>	30 202.19
<code>ceil(<i>x</i>)</code>	The ceiling of <i>x</i> : the smallest integer not less than <i>x</i> .	<code>math.ceil(-30.6)</code> <code>math.ceil(20.12)</code>	-30.0 21
<code>exp(<i>x</i>)</code>	The exponential of <i>x</i> : e^x	<code>math.exp(4)</code>	54.5981500331
<code>fabs(<i>x</i>)</code>	The absolute value of <i>x</i> .	<code>math.fabs(-30)</code> <code>math.fabs(202.19)</code>	30.0 202.19
<code>floor(<i>x</i>)</code>	The floor of <i>x</i> : the largest integer not greater than <i>x</i> .	<code>math.floor(-30.6)</code> <code>math.floor(20.12)</code>	-31.0 20.0
<code>log(<i>x</i>)</code>	The natural logarithm of <i>x</i> , for $x > 0$.	<code>math.log(10)</code>	2.30258509299
<code>log10(<i>x</i>)</code>	The base-10 logarithm of <i>x</i> , for $x > 0$.	<code>math.log10(10)</code>	1.0
<code>max(<i>x</i>₁, <i>x</i>₂, ...)</code>	The largest of its arguments: the value closest to positive infinity.	<code>max(10, 40, 25)</code>	40
<code>min(<i>x</i>₁, <i>x</i>₂, ...)</code>	The smallest of its arguments: the value closest to negative infinity.	<code>min(10, 40, 25)</code>	10
<code>modf(<i>x</i>)</code>	The fractional and integer parts of <i>x</i> in a two-item tuple. Both parts have the same sign as <i>x</i> . The integer part is returned as a float.	<code>math.modf(10.2)</code>	(0.1999999999999993, 10.0)
<code>pow(<i>x</i>, <i>y</i>)</code>	<i>x</i> to the power of <i>y</i> .	<code>math.pow(10, 2)</code>	100.0
<code>round(<i>x</i> [, <i>n</i>])</code>	<i>x</i> rounded to <i>n</i> digits from the decimal point.	<code>round(10.23167, 2)</code> <code>round(10.23167, 4)</code>	10.23 10.2317
<code>sqrt(<i>x</i>)</code>	The square root of <i>x</i> , for $x > 0$.	<code>math.sqrt(100)</code>	10.0

Table 13: Mathematical functions in Python

7.4. Trigonometric functions

Table 14 shows the trigonometric functions in Python. Note that the functions are not accessible directly. We need to import the math module first in order to call the function using the math static object, e.g., `math.acos(0)`.

Trigonometric functions	Description	Example	Output example
<code>acos(x)</code>	This function returns the arc cosine of x in radians.	<code>math.acos(0)</code>	1.57079632679
<code>asin(x)</code>	This function returns the arc sine of x in radians.	<code>math.asin(0)</code>	0.0
<code>atan(x)</code>	This function returns the arc tangent of x in radians.	<code>math.atan(0)</code>	0.0
<code>atan2(y, x)</code>	This function returns <code>atan(y/x)</code> in radians.	<code>math.atan2(5, 5)</code>	0.785398163397
<code>cos(x)</code>	This function returns the cosine of x in radians.	<code>math.cos(0)</code>	1.0
<code>degrees(x)</code>	This function converts angle x from radians to degrees.	<code>math.degrees(0)</code> <code>math.degrees(math.pi)</code>	0.0 180.0
<code>hypot(x, y)</code>	This function returns the Euclidean norm, <code>sqrt(x * x + y * y)</code> .	<code>math.hypot(0, 3)</code>	3.0
<code>sin(x)</code>	This function returns the sine of x in radians.	<code>math.sin(0)</code>	0.0
<code>tan(x)</code>	This function returns the tangent of x in radians.	<code>math.tan(0)</code>	0.0
<code>radians(x)</code>	This function converts angle x from degrees to radians.	<code>math.radians(0)</code>	0.0

Table 14: Trigonometric functions

7.5. Random number functions

Table 15 shows some random number functions which are commonly used in Python for, e.g., in games, privacy applications, security, testing, and simulation. Note that the functions are not accessible directly. We need to import the random module first in order to call the function using the random static object, e.g., `random.choice([1, 2, 3])`.

Random number function	Description	Example	Output example
<code>choice(seq)</code>	This function returns a random item from a list, tuple, or string.	<code>random.choice([1, 2, 3])</code> <code>random.choice('Hello')</code>	2 e
<code>random()</code>	This function returns a random float r , such that $0 \leq r < 1$	<code>random.random()</code>	0.19661692033
<code>randrange([start,] stop [,step])</code>	This function returns a randomly selected element from <code>range(start, stop, step)</code>	<code>random.randrange(0, 100, 5)</code>	50
<code>seed([x])</code>	This function sets the integer starting value used in generating random numbers. The method should be called before any other random module function is called. x is the seed for the next random number. The method takes system time to generate next random number if x is omitted.	<code>random.seed(5)</code> <code>random.random()</code> <code>random.seed(5)</code> <code>random.random()</code> <code>random.seed(5)</code> <code>random.random()</code>	0.62290169489 0.62290169489 0.62290169489
<code>shuffle(lst)</code>	This function randomizes the items of a list in place.	<code>list = [4, 56, 20, 34]</code> <code>random.shuffle(list)</code> <code>print (list)</code>	[56, 4, 34, 20]
<code>uniform(x, y)</code>	This function returns a random float r , such that $x \leq r < y$	<code>random.uniform(10, 25)</code>	18.6310645777

Table 15: Random number functions

8. Data types

Data types are an important concept in almost all programming languages. They represent a type of the data which can be processed in a computer program. Data types tell the interpreter how the programmer intends to use the data. It is important to specify the type of data when we write a computer program to process different types of data such as strings and integers. If we lack to specify the type of data, the computer will not understand how the different operations should be performed on the given data. The Python interpreter can determine which data type we are storing. So, there is no need to specify the data type in Python as it understands a given data type automatically. Many data types are available in Python. Some important ones we list below.

- Numbers (int, float, long, complex)
- Sequences (strings, bytes/byte array, lists, tuples)
- Boolean (true/false)
- Sets
- Dictionaries
- Module
- Function
- Class
- Method
- File

9. Variables

In simplest terms, we could say that a variable is just a box that you can use to put stuff in. To distinguish the different boxes (i.e., variables) we label the boxes. This means every variable has a name which describes what the variable is storing. We can view the content of the box and we can change the content just by calling the box label.

Formally, we can say that variables are reserved memory locations to store values (e.g., a letter or a number). When we create a variable we reserve some space in memory. These variables hold values temporarily during program execution. Based on the data type of a variable, the Python interpreter allocates memory and decides what can be stored in the reserved memory. Different data types like integers, decimals, characters, etc. can be stored in these variables.

Variables do not need to be declared in Python as the Python interpreter determines what type of data is stored. To assign values to a variable we use an equal sign (=) in the Python programming language. The equal sign assigns the value of right side operand to left side operand. The left side operand is the name of the variable and the right side operand is the assigned value.

Example

```
name = "John"           # A string
height = "200"          # An integer
distance = "4.5"        # A floating point
```

Example 6: Assigning values to variables

To create variable names in Python we should comply with the following rules:

- Variable names must begin with a letter or underscore.
- Variable names are case-sensitive.
- A variable name does not contain spaces.
- A variable name can only contain alphanumeric characters (i.e., a-z, A-Z, and 0-9) and underscore (_).
- Reserved keywords cannot be used as variable names.

10. Lists

The lists data structure in Python is used to organize data in a single set. Lists are sequences, just like tuples. Each element of a sequence is assigned a number, i.e., the index number or the position of the element. The first index number is zero, the second index number is one, the third index number is two, and so forth. On all sequence types (e.g., lists or tuples) we can apply operations like indexing, adding, multiplying, slicing, and checking for membership.

10.1. Creating lists

A list can be created by putting different comma-separated values or elements between square brackets.

Example

```
list1 = ['John', 'Paul', 'Catherine']
list2 = [100, 300, 500]
list3 = [20, 10, "Tina", "John"]
```

Example 7: Creating a list

10.2. Accessing values in lists

Values in a list can be accessed by square brackets for slicing along with index numbers. For example, $L[i]$ represents the value at index i in list L .

Example

```
list1 = ['John', 'Paul', 'Catherine']
list2 = [100, 300, 500]
list3 = [20, 10, "Tina", "John"]

print ("list1[0]=", list1[0])
print ("list2[2]=", list2[2])
print ("list3[1:3]=", list3[1:3])
```

Example 8: Accessing values in lists

Output example

```
list1[0]= John
list2[2]= 500
list3[1:3]= [10, 'Tina']
```

Table 16: Output of Example 8

10.3. Updating lists

Example 9 shows how single or multiple values can be updated in a list.

Example

```
list1 = ['John', 'Paul', 'Catherine']

print ("The third value in list1 is:")
print (list1[2])

print ("The value Catherine is updated in list1 with Carin:")
list1[2]= 'Carin'
print (list1)
```

Example 9: Updating and adding values to a list

Output example

```
The third value in list1 is:
Catherine
The value Catherine is updated in list1 with Carin:
['John', 'Paul', 'Carin']
```

Table 17: Output of Example 9

10.4. Deleting values from lists

The del-statement can be used to remove a value from a list by *del list_name[index]*.

Example

```
list1 = ['John', 'Paul', 'Catherine']

print ("list1 before deleting values:")
print (list1)
del list1[2]

print ("list1 after deleting value at index 2:")
print (list1)
```

Example 10: Deleting values from lists

Output example

```
list1 before deleting values:  
['John', 'Paul', 'Catherine']  
list1 after deleting value at index 2:  
['John', 'Paul']
```

Table 18: Output of Example 10

10.5. Basic list operations

The operators * (asterisk) and + (plus sign) work in lists almost the same as in strings. The * stands for repetition and the + stands for concatenation. The result is a new list instead of a string. Table 19 shows the basic list operations in Python.

Basic list operations	Description	Example	Output example
<code>len([x1, x2, x3,...])</code>	Length	<code>len([1, 2, 3, 4])</code>	4
<code>[x1, x2,..] + [x3, x4,..]</code>	Concatenation	<code>[1, 2] + [3, 4]</code>	<code>[1, 2, 3, 4]</code>
<code>[x] * y</code>	Repetition	<code>['hello'] * 3</code>	<code>['hello', 'hello', 'hello']</code>
<code>y in [x, y, z]</code>	Membership	<code>2 in [1, 2, 3, 4]</code>	True
<code>for x in [y1, y2,...]: print (x, end=" ")</code>	Iteration	<code>for x in [1, 2, 3]: print (x, end=" ")</code>	1 2 3

Table 19: Basic list operations in Python

10.6. Built-in list functions

Python has some built-in list functions which are shown in Table 20.

List functions	Description	Example	Output example
<code>len(list)</code>	This function returns the number of elements in the list.	<code>list = [1, 2, 3] print (len(list))</code>	3
<code>max(list)</code>	This function returns the elements from the list with	<code>list = [10, 280, 38] print (max(list))</code>	280

	maximum value.		
min(list)	This function returns the elements from the list with minimum value.	list = [10, 280, 38] print (min(list))	10
list(seq)	This function converts a tuple into list.	tuple = ('x', 1, 'r', 10) list = list(tuple) print (list)	['x', 1, 'r', 10]

Table 20: Built-in list functions in Python

10.7. Built-in list methods

Table 21 shows the built-in list methods in Python.

List methods	Description	Example	Output example
list.append(obj)	This method appends an object <i>obj</i> to <i>list</i> .	list1 = [1, 2] list1.append(3) print (list1)	[1, 2, 3]
list.count(obj)	This method returns count of how many times <i>obj</i> occurs in <i>list</i> .	list1 = [1, 2, 1, 1] print (list1.count(1))	3
list.extend(seq)	This method appends the contents of <i>seq</i> to <i>list</i> .	list1 = [1, 2] list2 = [3, 4] list1.extend(list2) print (list1)	[1, 2, 3, 4]
list.index(obj)	This method returns the lowest index in <i>list</i> that <i>obj</i> appears.	list1 = ['Zara', 'John', 'Joe'] print (list1.index('John'))	1
list.insert(index, obj)	This method inserts object <i>obj</i> into <i>list</i> at offset <i>index</i> .	list1 = ['a', 'b', 'd', 'e'] list1.insert(2, 'c') print (list1)	['a', 'b', 'c', 'd', 'e']
list.pop(obj=list[-1])	This method removes and returns last object or <i>obj</i> from <i>list</i> .	list1 = ['a', 'b', 'd', 'e'] print (list1.pop()) print (list1.pop(2))	e d
list.remove(obj)	This method removes object <i>obj</i> from <i>list</i> .	list1 = ['a', 'b', 'c', 'd', 'e'] list1.remove('c') print (list1)	['a', 'b', 'd', 'e']
list.reverse()	This method reverses objects of <i>list</i> in place.	list1 = ['a', 'b', 'c', 'd', 'e'] list1.reverse() print (list1)	['e', 'd', 'c', 'b', 'a']
list.sort([func])	This method sorts objects of <i>list</i> ; use compare func if given.	list1 = [34, 23, 'd', 'a', 'j'] list1.sort() print (list1)	[23, 34, 'a', 'd', 'j']

Table 21: Built-in list methods in Python

11. Tuples

Just like lists, tuples are sequences of objects which are immutable. Once a tuple is created it cannot be changed. The differences between tuples and lists are:

- tuples cannot be changed unlike lists,
- tuples are enclosed within parentheses instead of square brackets,
- values/elements of the tuples must have a defined order.

Like in lists, in tuples each element of a sequence is assigned a number, i.e., the index number or the position of the element. The first index number is zero, the second index number is one, the third index number is two, and so forth. We can apply operations like indexing, adding, multiplying, slicing, and checking for membership on tuples and other sequence types (e.g., lists).

11.1. Creating tuples

A tuple can be created by putting different comma-separated values or elements between parentheses.

Example

```
list1 = ('John', 'Paul', 'Catherine')
list2 = (100, 300, 500)
list3 = (20, 10, "Tina", "John")
```

Example 11: Creating a tuple

11.2. Accessing values in tuples

Values in a tuple can be accessed by square brackets for slicing along with index numbers. For example, $T[i]$ represents the value at index i in tuple T .

Example

```
tuple1 = ('John', 'Paul', 'Catherine')
tuple2 = (100, 300, 500)
tuple3 = (20, 10, "Tina", "John")

print ("tuple1[0]=", tuple1[0])
print ("tuple2[2]=", tuple2[2])
print ("tuple3[1:3]=", tuple3[1:3])
```

Example 12: Accessing values in tuples

Output example

```
tuple1[0]= John  
tuple2[2]= 500  
tuple3[1:3]= (10, 'Tina')
```

Table 22: Output of Example 12

11.3. Updating tuples

Tuples are immutable which means that we cannot update or change the values of tuple elements. However, we can join tuples to create a new tuple as Example 13 shows.

Example

```
tuple1 = ('John', 'Paul', 'Catherine')  
tuple2 = (23, 67, 78)  
tuple3 = tuple1 + tuple2  
  
print (tuple3)
```

Example 13: Joining tuples

Output example

```
('John', 'Paul', 'Catherine', 23, 67, 78)
```

Table 23: Output of Example 13

11.4. Deleting values from tuples

We cannot remove individual values from a tuple. But we can remove a whole tuple with the **del**-statement by *del tuple_name*.

Example

```
tuple1 = ('John', 'Paul', 'Catherine')  
  
del tuple1
```

Example 14: Deleting a tuple in Python

11.5. Basic tuple operations

The operators * (asterisk) and + (plus sign) work in tuples almost the same as in strings. The * stands for repetition and the + stands for concatenation. The result is a new tuple instead of a string. Table 24 shows the basic tuple operations in Python.

Basic tuple operations	Description	Example	Output example
<code>len((x1, x2, x3,...))</code>	Length	<code>len((1, 2, 3, 4))</code>	4
<code>(x1, x2,...) + (x3, x4,...)</code>	Concatenation	<code>(1, 2) + (3, 4)</code>	<code>(1, 2, 3, 4)</code>
<code>(x) * y</code>	Repetition	<code>('hello') * 3</code>	<code>('hello', 'hello', 'hello')</code>
<code>y in (x, y ,z)</code>	Membership	<code>2 in (1, 2, 3, 4)</code>	True
<code>for x in (y1, y2,...): print (x, end=" ")</code>	Iteration	<code>for x in (1, 2, 3): print (x, end=" ")</code>	1 2 3

Table 24: Basic tuple operations in Python

11.6. Built-in tuple functions

Table 25 shows the built-in tuple functions in Python.

Tuple functions	Description	Example	Output example
<code>len(tuple)</code>	This function returns the number of elements in the tuple.	<code>tuple = (1, 2, 3) print (len(tuple))</code>	3
<code>max(tuple)</code>	This function returns the elements from the tuple with maximum value.	<code>tuple = (10, 280, 38) print (max(tuple))</code>	280
<code>min(tuple)</code>	This function returns the elements from the tuple with minimum value.	<code>tuple = (10, 280, 38) print (min(tuple))</code>	10
<code>tuple(seq)</code>	This function converts a list into tuple.	<code>list = ['x', 1, 'r', 10] tuple = tuple(list) print (tuple)</code>	<code>('x', 1, 'r', 10)</code>

Table 25: Built-in tuple functions in Python

12. Dictionary

Dictionaries are the fundamental data structure in Python. They have been heavily optimized for memory overhead and lookup speed efficiency. A dictionary consists of keys and values, where each key is unique and maps a value. The combination of a key and its value is called a *key-value pair* or *item*. Each key is separated from its value by a colon (:) and the items are separated by a comma. A dictionary is enclosed by curly braces ({}). The keys of a dictionary should be of an immutable data type such as strings, numbers, or tuples. The values can be of any types.

12.1. Creating dictionaries

Example 15 shows a simple way to create a dictionary in Python.

Example

```
dict = {'Name': 'Catherine', 'Age': 18, 'Gender': 'Female'}
```

Example 15: Creating a dictionary

12.2. Accessing values in dictionaries

You can access values in a dictionary by using square brackets along with the key name to obtain the value.

Example

```
dict = {'Name': 'Catherine', 'Age': 18, 'Gender': 'Female'}  
print ("Name: ", dict['Name'])  
print ("Age: ", dict['Age'])  
print ("Gender: ", dict['Gender'])
```

Example 16: Accessing values in a dictionary

Output example

```
Name: Catherine  
Age: 18  
Gender: Female
```

Table 26: Output of Example 16

12.3. Updating dictionaries

A dictionary can be updated by

- adding a new key-value pair or a new entry,
- modifying an existing entry, or
- deleting an existing entry.

Example 17 shows how you can update an existing entry and how you can add an entry in a dictionary.

Example

```
dict = {'Name': 'Catherine', 'Age': 18, 'Gender': 'Female'}
dict['Age'] = 20                # update an existing entry
dict['Residence'] = "Amsterdam" # Add a new entry

print ("Name: ", dict['Name'])
print ("Age: ", dict['Age'])
print ("Gender: ", dict['Gender'])
print ("Residence: ", dict['Residence'])
```

Example 17: Updating dictionaries

Output example

```
Name: Catherine
Age: 20
Gender: Female
Residence: Amsterdam
```

Table 27: Output of Example 17

12.4. Deleting dictionaries

You can either remove individual entries from a dictionary with the **del**-statement or clear the entire contents of a dictionary with the **dict.clear()** method. The entire dictionary can be deleted with the **del**-statement by *del dictionary_name*. See also Example 18.

Example

```
dict = {'Name': 'Catherine', 'Age': 18, 'Gender': 'Female'}
del dict['Age']          # remove entry with key 'Age'
dict.clear()             # remove all entries in dict
del dict                 # delete entire dictionary
```

Example 18: Deleting entries from dictionaries

12.5. Built-in dictionary functions

Python has some built-in dictionary functions which are shown in Table 28.

Dictionary functions	Description	Example	Output example
<code>len(dict)</code>	This function returns the number of items in the dictionary.	<code>dict = {'Name': 'Jean', 'Age': 17}</code> <code>print (len(dict))</code>	2
<code>str(dict)</code>	This function produces a printable string representation of a dictionary.	<code>dict = {'Name': 'Jean', 'Age': 17}</code> <code>print (str(dict))</code>	<code>{'Name': 'Jean', 'Age': 17}</code>
<code>type(variable)</code>	This function returns the type of the passed variable. If the passed variable is dictionary then it would return a dictionary type.	<code>dict = {'Name': 'Jean', 'Age': 17}</code> <code>print (type(dict))</code>	<code><type 'dict'></code>

Table 28: Built-in dictionary functions in Python

12.6. Built-in dictionary methods

Table 29 shows the built-in dictionary methods in Python.

Dictionary methods	Description	Example	Output example
<code>dict.clear()</code>	This method removes all elements of dictionary <code>dict</code> .	<pre>dict = {'Name': 'Tina', 'Age': 18} dict.clear() print (dict)</pre>	<code>{ }</code>
<code>dict.copy()</code>	This method returns a shallow copy of dictionary <code>dict</code> .	<pre>dict1 = {'Name': 'Tina', 'Age': 18} dict2 = dict1.copy() print (str(dict2))</pre>	<code>{'Name': 'Tina', 'Age': 18, }</code>
<code>dict.fromkeys(seq[, value])</code>	This method creates a new dictionary with keys from <i>seq</i> and <i>values</i> set to <i>value</i> if provided.	<pre>seq = ('Name', 'Age') dict = dict.fromkeys(seq, 20) print (str(dict))</pre>	<code>{'Name': 20, 'Age': 20}</code>
<code>dict.get(key, default=None)</code>	This method returns a value for the given key. <i>Default</i> is the value that is returned in case the key does not exist.	<pre>dict = {'Name': 'Tina', 'Age': 18} print (dict.get('Name')) print (dict.get('Gender', "Does not exist"))</pre>	Tina Does not exist
<code>dict.items()</code>	This method returns a view of dict's (key, value) tuple pairs	<pre>dict = {'Name': 'Tina', 'Age': 18} print (dict.items())</pre>	<code>dict_items([('Name', 'Tina'), ('Age', 18)])</code>
<code>dict.keys()</code>	This method returns a view of all available keys in the dictionary.	<pre>dict = {'Name': 'Tina', 'Age': 18} print (dict.keys())</pre>	<code>dict_keys(['Name', 'Age'])</code>
<code>dict.setdefault(key, default=None)</code>	This method is similar to <code>get()</code> , but will set <code>dict[key]=default</code> if key is not already in dict.	<pre>dict = {'Name': 'Tina', 'Age': 18} print (dict.setdefault ('Name', "None")) print (dict.setdefault ('Gender', "None")) print ("New dict: %s" % dict)</pre>	Tina None New dict: {'Name': 'Tina', 'Age': 18, 'Gender': 'None'}
<code>dict.update(dict2)</code>	This method adds dictionary <code>dict2</code> 's key-values pairs to <code>dict1</code> .	<pre>dict1 = {'Name': 'Tina', 'Age': 18} dict2 = {'Gender': 'female'} dict1.update(dict2) print (dict1)</pre>	<code>{'Name': 'Tina', 'Age': 18, 'Gender': 'female'}</code>
<code>dict.values()</code>	This method returns a view of all the values in a given dictionary.	<pre>dict = {'Name': 'Tina', 'Age': 18} print (dict.values())</pre>	<code>dict_values(['Tina', 18])</code>

Table 29: Built-in dictionary methods in Python

13. Decision-making

In real life, we encounter sometimes situations when we need to make decisions. Based on these decisions we decide what to do next. Likewise, we encounter situations in programming where we need to make some decisions. Based on these decisions we execute the next block of code.

Flowcharts are very helpful in understanding and recognizing decision making structures. The following decision-making statements are available in Python.

- if-statements
- if...else statements
- if...elif...else statements
- nested if-statements

13.1. if-statement

If-statements consists of a Boolean expression which evaluates to TRUE or FALSE (see Figure 1). If the Boolean expression evaluates to TRUE, then the block of statement(s) inside the if-statement is executed. If the Boolean expression evaluates to FALSE, then the first set of code after the end of the if-statement(s) is executed.

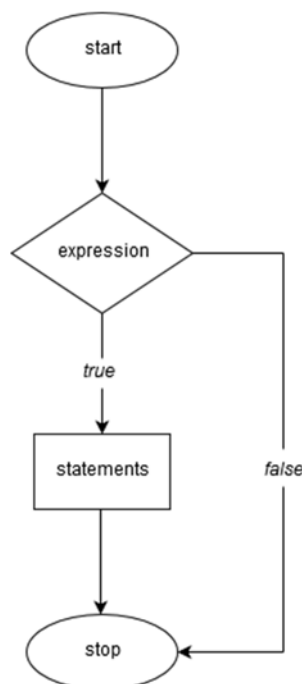


Figure 1: if-statement depicted in a flowchart

Example 19 shows two situations of an if-statement code. In the first situation, the Boolean expression evaluates to TRUE and in the other situation to FALSE.

Example

```
# Boolean evaluates to TRUE
x = 10
if x > 5:
    print("x is greater than 5")

# Boolean evaluates to FALSE
y = 10
if y > 100:
    print("y is greater than 100")
print("Bye bye!")
```

Example 19: if-statement in Python

13.2. if...else-statement

In the *if...else-statement*, the if-statement is followed by an optional else-statement. The if...else-statement contains a Boolean expression (see Figure 2) which if evaluates to FALSE, the else-statement is executed.

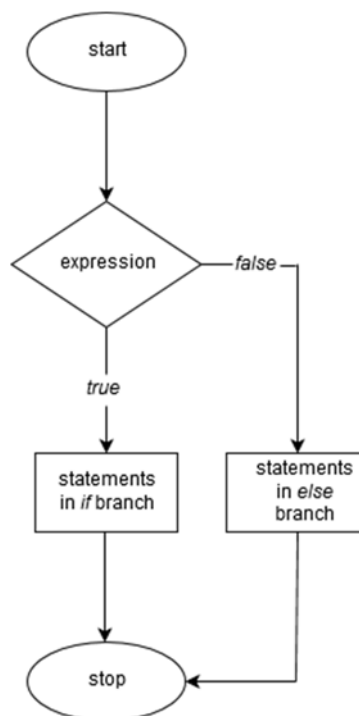


Figure 2: if...else-statement depicted in a flowchart

Example 20 shows an if...else-statement code in which the Boolean expression is evaluated to FALSE which results in the execution of the else-statement.

Example

```
x = 1
if x > 5:                                # Boolean evaluates to TRUE
    print("x is greater than 5")
else:                                    # Boolean evaluates to FALSE
    print("x is less than 5")
```

Example 20: if...else-statement in Python

13.3. if...elif...else-statement

To check multiple expressions we use the *if...elif...else-statement*. *Elif* is short for *else if*. It is also called the *chained conditional statement* (see Figure 3). The condition of the next elif-statement is checked if the condition for the if-statement is FALSE. If the condition of the next elif-statement is FALSE, then the condition of the next elif-statement is checked and so on. If all the conditions are FALSE, the else-statement is executed. The if...elif...else-statement can have only one else-statement and multiple elif-statements. Note that only one elif statement is executed according to the condition.

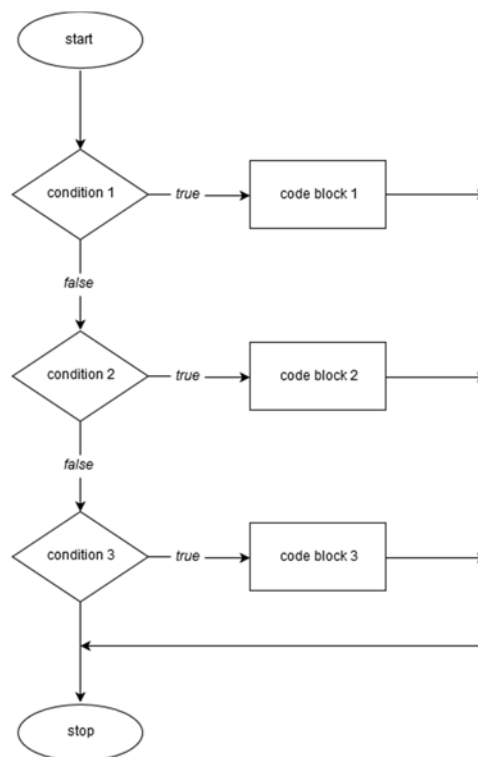


Figure 3: if...elif...else-statement depicted in a flowchart

Example 21 shows an if...elif...else-statement code in which the condition of the elif-statement is evaluated to TRUE which results in the execution of the elif-statement.

Example

```
x = 1
y = 5

if x > y:
    print("x is greater than y")
elif x < y:
    print("x is less than y")
else:
    print("x is equal to y")
```

Example 21: if...elif..else-statement in Python

13.4. Nested if-statement

Sometimes there may be situations in which you want to check for another condition after a condition evaluates to TRUE. In such situations, you can use nested if-statements. This is called *nesting* in computer programming. In nested if-statements, you can have an if...elif...else-statement inside another if...elif...else-statement. Nesting should be avoided when possible, as indentation is the only way to figure out the level of nesting in Python which can get confusing.

Example 22 shows a nesting code in Python in which the condition of the elif-statement is evaluated two times to TRUE before the program quits.

Example

```
x = 10

if x < 11:
    print("x is less than 11")
    if x == 5:
        print("x is 5")
    elif x > 5:
        print("x is greater than 5")
    elif x < 5:
        print("x is less than 5")
else:
    print("Cannot guess x")
print("Leaving program")
```

Example 22: Nesting in Python

14. Loops

In computer programming, a *loop* is a sequence of instructions that is repeated until a certain condition is met. The loop iteration technique is used to repeat the same or similar type of tasks based on a specified condition. Two of the most common types of loops are the *while-loop* and the *for-loop*. Besides the while-loops and the for-loops, we will also discuss here the nested loop.

14.1. While-loop

The while-loop is the simplest form of a programming loop. The statements in a while-loop are repeated as long as a given condition is TRUE (see Figure 4).

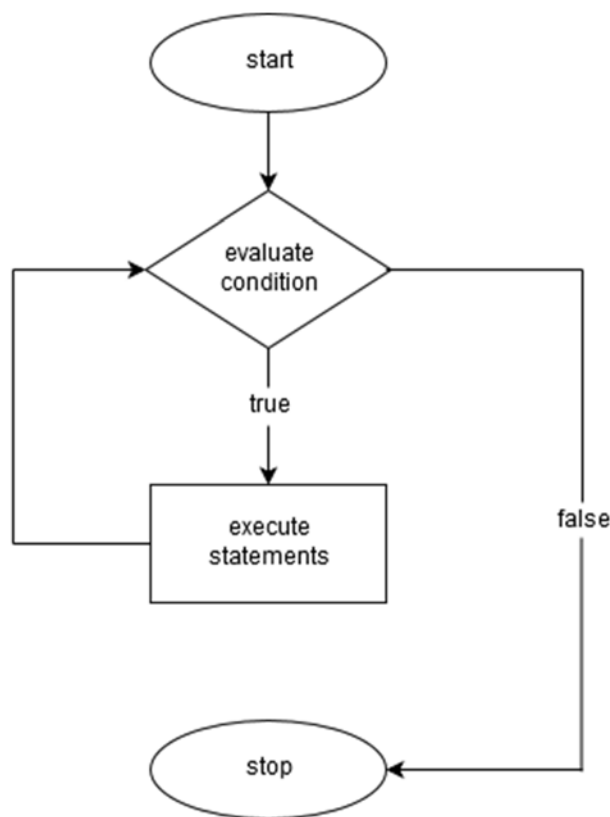


Figure 4: while-loop depicted in a flowchart

Example 23 shows loop created with the while-loop. Note that x is incremented with one after every iteration, or else the loop will continue forever.

Example

```
x = 1
while x < 5:
    print("Iteration:", x)
    x = x + 1
```

Example 23: while-loop in Python

Output example

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
```

Table 30: Output of Example 23

14.2. For-loop

The for-loop is used when we want to repeat a piece of code 'n' times (see Figure 5) or when we want to iterate over a sequence (i.e., a list, a tuple or a string).

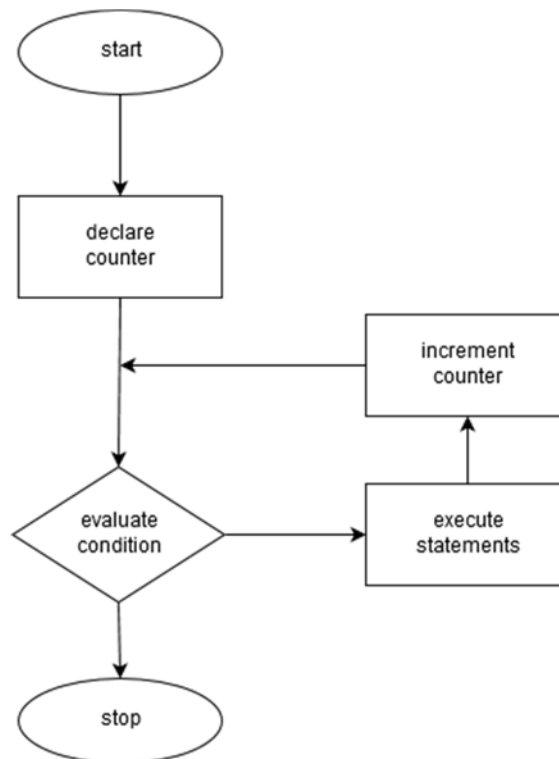


Figure 5: for-loop depicted in a flowchart

Example 24 shows how to repeat a piece of code ‘n’ times with a for-loop.

Example

```
for x in range (0,4):  
    print("Iteration:", x)
```

Example 24: repeat ‘n’ times with a for-loop in Python

Output example

```
Iteration: 0  
Iteration: 1  
Iteration: 2  
Iteration: 3
```

Table 31: Output of Example 24

Example 25 shows how to iterate over a sequence with a for-loop in Python.

Example

```
fruits = ["apple", "banana", "pineapple"]  
for i in fruits:  
    print(i)
```

Example 25: Iterate over a sequence with a for-loop in Python

Output example

```
apple  
banana  
pineapple
```

Table 32: Output of Example 25

14.3. Nested loop

We talk about nested loops when we use a loop inside another loop, e.g., a for-loop inside a while-loop or vice versa. We can also put a for-loop inside a for-loop and a while-loop inside a while-loop. Example 26 gives an example of a nested loop using a for-loop inside a for-loop.

Example

```
for g in range(1, 3):
    for k in range(1, 3):
        print ("%d * %d = %d" % (g, k, g*k))
```

Example 26: A nested loop in Python

Output example

```
1 * 1 = 1
1 * 2 = 2
2 * 1 = 2
2 * 2 = 4
```

Table 33: Output of Example 26

14.4. Loop control statements

A loop control statement is a statement that determines whether other statements will be executed or not. The Python language supports three loop control statements:

- break statement,
- continue statement, and
- pass statement.

14.4.1. Break statement

With the break statement we can stop the loop before it has looped through all the statements. It terminates the loop statement and transfers execution to the statement next to the loop. The break statement can be used in both while-loops and for-loops.

Example

```
fruits = ["apple", "banana", "pineapple"]
for i in fruits:
    if i == "banana":
        break
    print(i)
```

Example 27: break statement in Python

Output example

```
apple
```

Table 34: Output of Example 27

14.4.2. Continue statement

With the continue statement we can stop the current iteration of the loop, and continue with the next. The continue statement can be used in both while-loops and for-loops.

Example

```
fruits = ["apple", "banana", "pineapple"]
for i in fruits:
    if i == "banana":
        continue
    print(i)
```

Example 28: continue statement in Python

Output example

```
apple
pineapple
```

Table 35: Output of Example 28

14.4.3. Pass statement

The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute. Nothing will happen when the pass statement is used. It is therefore called a *null* operation. The pass statement is especially useful in places where your code will eventually go but has not been written yet.

Example

```
for letter in 'Python':
    if letter == 'h':
        pass
    print("pass block")
    print(letter)
```

Example 29: pass statement in Python

Output example

```
P
y
t
pass block
h
o
n
```

Table 36: Output of Example 29

15. Functions

A function is a block of code which runs when it is called. Functions make codes reusable. We can pass data into a function and a function can return data as a result. The Python programming language has many built-in functions like `print()`, `random()`, and other built-in functions. It is also possible to create your own functions in Python. These functions are called *user-defined functions*.

15.1. Creating a function

In Python, a function block is started with the **def** keyword followed by the name of the function and parentheses. The code block within the function starts with a colon (:) and is indented (see Example 30).

Example

```
def my_function():  
    print("Hello World")
```

Example 30: Creating a function in Python

15.2. Calling a function

We can call a function just by using the function name followed by parenthesis (see Example 31).

Example

```
def my_function():  
    print("Hello World")  
  
my_function()
```

Example 31: Calling a function in Python

15.3. Parameters and arguments

Parameters are variables in a function definition. We pass data to functions as parameters which are specified after the function name inside the parenthesis. The parameters are separated with a comma, if the function has many parameters. An argument is the actual value of the parameter that gets passed to function. Example 32 shows a function with one parameter, called name. When the function is called, it prints hello and the name that is passed along. The name that is passed along is used inside the function.

Example

```
def my_function(name):  
    print("Hello " + name)  
  
my_function("John")  
my_function("Catherine")
```

Example 32: Parameters in Python

Example 33 shows how to use a default parameter value in a Python function. When the function is called without a parameter, the default value is used.

Example

```
def my_function(name = "World"):  
    print("Hello " + name)  
  
my_function("John")  
my_function("Catherine")  
my_function()
```

Example 33: Default parameter value in Python

15.4. Return values

With the **return** statement, we can let a function return a value (). This makes it possible to use a function output outside the function.

Example

```
def average(x, y):  
    avg = (x + y)/2  
    print("Inside: Average is: ", avg)  
    return avg  
  
avg = average(2, 6)  
# We can print avg outside the function  
# because it is returned by the function  
print("Outside: Average is: ", avg)
```

15.5. Parameters vs arguments

The term parameter is often used to refer to the variable names within a function. An argument can be thought of as the value that is assigned to that variable. Parameters are in fact placeholders within a function for the arguments which are passed. In Example 35, *name* is the parameter for the function *my_function*. Anywhere we see *name* within the function will act as a placeholder until *name* is passed as an argument. The argument is passed to the function by *my_function("John")*. This calls the function, *my_function*, and assigns the value of 'John' (pass the argument 'John') to the parameter *name*. Now, the parameter *name* within the function will act as a variable with the value of 'John'.

Example

```
def my_function(name):  
    print("Hello " + name)  
  
my_function("John")
```

Example 35: Parameter vs argument

16. Modules

A module can be considered to be the same as a code library, a file consisting of Python code, e.g., set of functions, that we want to include in your application. It allows us to organize the Python code logically and makes the code easier to understand and use. Module codes can be reused as many times as we require.

16.1. Creating modules

We can create a module by saving the Python code in a file with the file extension `.py`. To create a module of Example 36, we just have to save the code in a file, e.g., `my_module.py`.

Example

```
def greeting(name):  
    print("Hello" + name)
```

Example 36: Creating a module in Python

16.2. Using modules

16.2.1. Import statement

We can use a module by using the *import statement* followed by the name of the module(s). The module names are separated by a comma when we import multiple modules. The module created in 16.1 can be used by importing the module named `my_module` and calling the `greeting` function (see Example 37). Use the syntax *module_name.function_name* when you use a function from a module.

Example

```
import my_module  
  
my_module.greeting("John")
```

Example 37: Using modules in Python

16.2.2. Import...from statement

With the *import...from statement* it is possible to import a part of a module instead of the whole module. In this way, we can import specific attributes from a module into the current namespace. Suppose we have a module, named `my_module`, as shown in Example 38.

Example

```
def greeting(name):  
    print("Hello " + name)  
  
def gb(name):  
    print("Goodbye " + name)
```

Example 38: Module with multiple functions

To import only the *gb* function from the module *my_module*, we use the syntax *from module_name import name* (see Example 39).

Example

```
from my_module import gb  
  
gb("John")
```

Example 39: Import attributes from a module

Note that when import specific attributes from a module using the *from* keyword we do not use the module name when we refer to elements in the module, e.g., not *my_module.gb("John")*, but *gb("John")*.

16.3. Variables in modules

Modules also can contain variables of all types, like arrays, dictionaries, objects, etc. (see Example 40 for a module named *my_module*).

Example

```
person1 = {  
    "Name": "Catherine",  
    "Age": "24",  
    "Residence": "Amsterdam"  
}
```

Example 40: Variables in modules

We can access the *person1* dictionary by importing the module named *my_module* (see Example 41).

Example

```
import my_module

country = my_module.person1["Residence"]
print(country)
```

Example 41: Importing a dictionary from a module

16.4. Renaming modules

We can create an alias for a module by using the *as* keyword when we import a module (see Example 42).

Example

```
import my_module as MM

MM.greeting("John")
```

Example 42: Renaming modules in Python

16.5. Using the `dir()` function

The **`dir()`** built-in function returns a list of defined names belonging to a module (see Example 43). The list contains the names of all the modules, variables and functions that are defined in a module (see Table 37 for the output of Example 43). Note that the **`dir()`** function can be used on all modules, also on the ones we create ourselves.

Example

```
import math

list_of_names = dir(math)
print(list_of_names)
```

Example 43: Using the `dir()` function

Output example

```
['__doc__', '__name__', '__path__', 'acos', 'acosh', 'asin', 'asinh',  
'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh',  
'degrees', 'e', 'exp', 'fabs', 'factorial', 'floor', 'hypot', 'isinf',  
'isnan', 'log', 'log10', 'pi', 'pow', 'radians', 'sin', 'sinh',  
'sqrt', 'tan', 'tanh', 'trunc']
```

Table 37: Output of Example 43

17. Date & Time

There are several ways to handle date and time in a Python program. Python's time and calendar modules help track dates and times. Time intervals have to be represented in floating point numbers in units of seconds which are expressed by *tick* in python. The function **time()** from Python's time module returns the current system time in ticks since 12:00 am, January 1, 1970 (epoch).

Example

```
import time

ticks = time.time()
print ("Number of ticks:", ticks)
```

Example 44: Python's time module

Output example

```
Number of ticks: 1530196021.89
```

Table 38: Output of Example 44

Another way to work with time is using Python's **datetime** module (see Example 45). The method **strftime()** is used to format date objects into readable strings. This method takes one parameter, *format*, to specify the format of the returned string. Table 40 shows a reference of all the legal format codes for the strftime() method.

Example

```
from datetime import datetime

current = datetime.now()
print("Current date and time: ", current)
print("Today's date is: ", current.strftime("%Y-%m-%d"))
print("Year: ", current.year)
print("Month: ", current.month)
print("Day: ", current.day)
```

Example 45: Python's datetime module

Output example

```
Current date and time: 2018-06-28 16:46:25.420000
Today's date is: 2018-06-28
Year: 2018
Month: 6
Day: 28
```

Table 39: Output of Example 45

Directive	Description	Example
%a	Weekday, short version	Tue
%A	Weekday, full version	Tuesday
%w	Weekday as a number 0-6, 0 is Sunday	2
%d	Day of month 01-31	21
%b	Month name, short version	Nov
%B	Month name, full version	November
%m	Month as a number 01-12	11
%y	Year, short version, without century	17
%Y	Year, full version	2017
%H	Hour 00-23	13
%I	Hour 00-12	01
%p	AM/PM	PM
%M	Minute 00-59	31
%S	Second 00-59	09
%f	Microsecond 000000-999999	586512
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52
%W	Week number of year, Monday as the first day of week, 00-53	52
%c	Local version of date and time	Tue Nov 21 13:31:00 2017

%x	Local version of date	11/21/17
%X	Local version of time	13:31:00
%%	A % character	%

Table 40: Reference of all the legal format codes for the `strftime()` method

18. Files I/O

Programs need an input to process and an output to display data (see Figure 6). The input can come, e.g., from a keyboard, an output from another program or a file. The output can be sent, e.g., to a computer screen, a printer, or another program.



Figure 6: Input and output stream for a program

We can store the data in variables while a program runs. But this data get lost if we terminate the program. If we want to keep the data after the termination of the program, we have to store the data in a file. The `print` function converts the expressions you pass into a string and writes the result to standard output (`stdout`).

18.1. Writing to standard output

Writing to the standard output (`stdout`), e.g., a computer screen, can be done by the `print` statement.

Example

```
print ("Hello world")
```

Example 46: Writing to standard output

Output example

```
Hello world
```

Table 41: Output of Example 46

18.2. Reading from standard input

Reading from the standard input (*stdin*), e.g., a keyboard, can be done by Python's built-in function `input()`.

The **`input()`** function interprets the user's input. If the user, e.g., puts in an integer value, the input function returns this integer value. If the user, on the other hand, inputs a list, the function will return a list. A prompt is displayed to enter the string.

Example

```
str = input("Enter your input: ")
print ("Received input is: ", str)
```

Example 47: The use of `input()` function to read from standard input

Output example

```
Received input is: Hello world
```

Table 42: Output of Example 47 if we give "Hello world" as input

18.3. Opening a file

We have to use Python's built-in **`open()`** function to open a file before we can read or write a file. The `open()` function creates a *file object* and requires two arguments, the file name, and the file opening mode. Table 43 shows the different modes of opening a file in Python.

Mode	Description
t	This mode opens a file in text mode (default mode).
b	This mode opens a file in binary mode.
+	This mode opens a file for updating (reading and writing).
r	This mode opens a file for reading only (default mode).
rb	This mode opens a file for reading only in binary format.
r+	This mode opens a file for both reading and writing.

rb+	This mode opens a file for both reading and writing in binary format.
w	This mode opens a file for writing only. It overwrites the file if the file exists. If the file does not exist, it creates a new file for writing.
wb	This mode opens a file for writing only in binary format. It overwrites the file if the file exists. If the file does not exist, it creates a new file for writing in binary format.
w+	This mode opens a file for both writing and reading. It overwrites the file if the file exists. If the file does not exist, it creates a new file for writing and reading.
wb+	This mode opens a file for both writing and reading in binary format. It overwrites the file if the file exists. If the file does not exist, it creates a new file for writing and reading in binary format.
a	This mode opens a file for appending (pointer is at the end of the file). If the file does not exist, it creates a new file for writing.
ab	This mode opens a file for appending in binary format (pointer is at the end of the file). If the file does not exist, it creates a new file for writing in binary format.
a+	This mode opens a file for both appending and reading (pointer is at the end of the file). If the file does not exist, it creates a new file for reading and writing.
ab+	This mode opens a file for both appending and reading in binary format (pointer is at the end of the file). If the file does not exist, it creates a new file for reading and writing in binary format.
x	This mode creates a file. It returns an error if the file exists.

Table 43: Modes of opening a file in Python

We can get various information related to a file once the file is open by using attributes related to the file object. Table 44 shows the attributes which can be used with the file object.

Mode	Description
file.closed	This attribute returns TRUE if the file is closed and FALSE otherwise.
file.mode	This attribute returns the access mode with which file was opened.
file.name	This attribute returns name of the file.

Table 44: Attributes related to file object

Example 48 shows how we use the attributes on an open file.

Example

```
demo = open("demo.txt", "w")
print ("Name of the file: ", demo.name)
print ("Opening mode: ", demo.mode)
print ("Closed: ", demo.closed)
```

Example 48: Opening a file and the use of the related attributes

Output example

```
Name of the file: demo.txt
Opening mode: w
Closed: False
```

Table 45: Output of Example 48

18.4. Closing a file

A file that is open can be closed by the **close()** method.

Example

```
demo = open("demo.txt", "r")          # Open the demo file in reading mode
print ("Name of the file: ", demo.name)
demo.close()                          # Close the demo file
```

Example 49: Closing a file

18.5. Reading a file

We can read a file by the **read()** method once it is open by the open() function.

18.5.1. Read the whole file

Example 50 shows how we can read all the content of a file at once.

Example

```
demo = open("demo.txt", "r")
# Read all the content of the demo file
str = demo.read()
print(str)
```

Example 50: Read all the content of the file

18.5.2. Read a part of a file

Example 51 shows how we can read a part of a file.

Example

```
demo = open("demo.txt", "r")
# Return the 5 first characters of the demo file
str = demo.read(5)
print(str)
```

Example 51: Read a part of a file

18.5.3. Read a few lines of a file

Example 52 shows how we can read one line of a file with the ***readline()*** method. If we want to read, e.g., the first three lines of a file, we call the ***readline()*** method three times.

Example

```
demo = open("demo.txt", "r")
# Read one line of the demo file
str = demo.readline()
print(str)
```

Example 52: Read one line of a file

18.5.4. Looping through a file

Example 53 shows how we can loop through a file line by line. This is useful when we want to read a whole file line by line.

Example

```
demo = open("demo.txt", "r")
# Looping through the file
for x in demo:
    print(x)
```

Example 53: Looping through a file line by line

18.6. Writing a file

With the ***write()*** method we can write to an open file. Note that the ***write()*** method does not add a newline character (`'\n'`) to the end of a string.

Example 54 shows how we can add a new line to an existing file. The append mode ("a") adds strings to the end of a file.

Example

```
demo = open("demo.txt", "a")
demo.write("A new line is added to the file")
```

Example 54: Add a new line with the write() method

Example 55 shows how we can overwrite an existing file. The write mode ("w") overwrites any existing content.

Example

```
demo = open("demo.txt", "w")
demo.write("Existing content has been deleted")
```

Example 55: Overwriting an existing file

18.7. Creating a file

Three modes can be used to create a file with the open() method, namely with the modes create("x"), append("a"), and write("w").

- create("x"), it returns an error if the file exists.
- append("a"), file is only created if the specified file does not exist.
- write("w"), file is only created if the specified file does not exist.

Example 56 shows how a file named, demo.txt, is created if the file does not exist.

Example

```
demo = open("demo.txt", "w")
```

Example 56: Create the demo file

18.8. Deleting a file or a directory

We have to import the **os module** to delete a file or a directory. This module provides us methods to perform file processing operations, such as deleting and renaming files. To delete a file we use the **remove()** method. With the **rmdir()** method we can delete empty directories

(folders). In Example 57 we check whether the file demo.txt exists or not. We delete the file if it exists otherwise we print “File not exist”.

Example

```
import os

# Check if the file exists
if os.path.exists("demo.txt"):
    os.remove("demo.txt")
    print("File deleted")
else:
    print("File does not exist")
```

Example 57: Delete a file

18.9. Directory methods

Example 58 shows some commonly used directory methods. We need to import the os module first in order to call the methods.

Method	Description	Example
chdir(path)	This method is used to change the current directory.	os.chdir("/misc/tmp")
getcwd()	This method displays the current working directory.	os.getcwd()
makedirs(path[,mode])	This method creates a directory.	os.makedirs("/misc/tmp")
rename(src, dst)	This method renames the file or directory <i>src</i> to <i>dst</i> .	os.rename("tut", "tutorial")
rmdir(path)	This method deletes an empty directory.	os.rmdir("/misc/tmp")

Example 58: Some directory methods

Index

A

argument45

B

break statement42

C

chained conditional statement.....37

close()58

continue statement43

I

import statement48

import...from statement.....48

input56

K

key-value pair31

L

loop39

N

nested if-statements.....38

nesting38

numerical data types17

O

open()56

Operand10

Operator10

os module60

P

pass statement43

Python7

python interactive mode8

Python interpreter8

python script mode.....8

R

read()58

readline()59

remove()60

reserved keywords.....9

rmdir()60

S

standard input56

standard output.....55

stdin See standard input

stdout..... See standard output

string15

T

tick52

U

user-defined functions.....45

W

while loop39

write()59

Acknowledgements

Python (<https://www.python.org>)

Trademarks

Python and PyCon are trademarks or registered trademarks of the Python Software Foundation.

Licenses

Python, its standard libraries, and Jython, are distributed under the Python License. The intellectual property rights behind Python and Jython are held and managed by the Python Software Foundation.

The licenses, trademarks, and copyrights for other implementations of Python (such as IronPython, Stackless Python, and PyPy) may vary and are managed by their respective owners.